

# Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows

Bardia Mohammadi<sup>1</sup>, Nearchos Potamitis<sup>2</sup>, Lars Klein<sup>3</sup>,  
Akhil Arora<sup>2</sup>, Laurent Bindschaedler<sup>1</sup>

<sup>1</sup>Max Planck Institute for Software Systems <sup>2</sup>Aarhus University <sup>3</sup>EPFL  
{bmohammadi, bindsch}@mpi-sws.org

## Abstract

LLM agents execute multi-step workflows that mutate external state through tools. Common orchestrators treat tool return as the settlement trigger, so faults, speculation, and concurrent agents can leave partial effects, losing-branch residue, stale writes, or irreversible sends. Correct settlement needs two facts that retries, checkpoint replay, locks, and compensation each conflate: which effects must settle together, and when earlier conflicting work is exhausted. **Atomix** makes this split explicit with progress-aware transactions. The runtime records reads and effects during execution, seals a transaction when its footprint is complete, and commits only after per-resource frontiers show that no earlier conflicting work can still arrive. Commit is final settlement: Atomix releases bufferable effects, accepts reversible external effects as final, and lets irreversible effects leave the gate. Abort suppresses unreleased effects and compensates externalized reversible effects where possible. On representative agent workloads, this composition improves clean recovery under injected faults, isolates contending and speculative work, and prevents correctly classified irreversible actions from leaking; microbenchmarks show microsecond-scale wrapper overhead relative to tool latency. The code and artifacts of Atomix are available at <https://github.com/mpi-dsg/atomix>.

## 1 Introduction

LLM agents execute multi-step tasks that mutate external state: editing code, updating databases, filling web forms, sending emails, and calling third-party APIs LangChain Inc. [2026], CrewAI, Inc. [2026], Wu et al. [2024], Anthropic [2026], OpenAI [2026], Model Context Protocol Project [2025]. These workloads increasingly use concurrent execution: multiple agents share resources, parallel plans explore alternatives, and speculative execution starts downstream work before upstream confirmation. Each pattern raises the same question: *when may a tool effect become permanent?*

Current agent frameworks expose this decision indirectly. Orchestrators provide checkpointing, retries, and timeouts, and rely on tool-level idempotency keys plus Saga-style compensations LangChain Inc. [2026], Temporal Technologies Inc. [2026], Amazon Web Services [2026], Garcia-Molina and Salem [1987]. These mechanisms recover after a tool effect has externalized or after the workflow restores its own state. That execution model leaves three gaps once workflows run in parallel:

1. *Speculation*: parallel plans execute real tool calls; losing branches leave residual side effects.
2. *Contention*: concurrent agents touch the same resource and interleave writes.
3. *Irreversibility*: some effects cannot be undone once externalized.

**Atomix** adds transactional settlement at the tool interface. Atomix separates execution, sealing, frontier checks, and settlement. During execution, adapters record the resources a transaction reads and the effects it may produce. The orchestrator then *seals* the transaction, declaring that no more reads, scopes, or effects will be added. Atomix commits a sealed transaction only when per-resource frontiers certify, under the orchestrator’s advancement rule, that no earlier work on any

touched resource can still arrive. Commit settles the transaction: Atomix releases staged effects, lets irreversible effects leave the gate, and accepts already-externalized reversible effects as final. Abort suppresses unreleased effects and compensates externalized reversible effects where possible. The design adapts Sagas, Try-Confirm-Cancel, and streaming watermarks to the LLM tool interface. It requires orchestrator cooperation for progress tracking, but tools need no modification when all calls pass through adapters.

**Why This Design Is Needed.** No single existing trigger both admits speculative execution and decides when effects may safely become external. If tool return triggers settlement, a losing speculative branch or a failed later step has already mutated the world. If checkpoint rollback is the recovery mechanism, the runtime restores agent memory but not an email, booking, or remote API update. Saga compensation helps only when every externalized effect is reversible and has a correct handler; it cannot prevent an irreversible send. Locks cover only part of the problem: per-call locks miss the read-plan-write gap, while workflow-wide locks serialize unrelated work. Optimistic revalidation detects some stale writes, but still needs a closed transaction footprint and a rule for when irreversible effects may leave the gate. Frontiers provide ordering, but they do not make effects atomic. Transactions group effects, but by themselves do not show that earlier conflicting work has finished. Atomix therefore combines transaction grouping, seal, per-resource progress, and effect-class settlement.

**Evaluation.** We evaluate fault recovery, frontier-gated isolation, and irreversible-effect gating, then combine them in a stress test. Under heavy fault injection on  $\tau$ -bench, Atomix retains 57% clean task success while every non-Checkpoint-Replay baseline falls to 0–7%. Under forced multi-agent overlap it records 0 conflict-cycle witnesses. On irreversible sends it leaks 0/500 invalid sends and releases all 500 valid ones. On combined stress, Atomix is Pareto-best on the joint surface: in the top run-clean tier with zero wait, zero rejected commits, and the same 17-LOC adapter declaration across effect classes, while every tied baseline pays on at least one of these axes.

#### Contributions.

- We define a transactional settlement model for agent tool effects that combines sealing, read/effect scopes, per-resource progress predicates, effect-class-aware commit, and shim-based adapter interposition (§2).
- We implement the model in a runtime that splits tool execution from transaction settlement through adapters, a transaction manager, and a progress tracker (§2; implementation details in Appendix A).
- We evaluate fault recovery, frontier-gated isolation, and irreversible-effect gating against recovery and isolation alternatives implemented in a common harness (§3).

**Scope.** Atomix is an execution-layer runtime that composes with, rather than replaces, information-flow control, planning-level validation, and kernel-level confinement (§5). We do not claim semantic validation, distributed deployment, or full crash-safe exactly-once. The prototype implements crash-safe deduplication but validates it only with unit tests.

## 2 Transactional Abstractions and Runtime Semantics

Atomix’s runtime model separates tool execution from effect settlement. We first define the failure modes, then the runtime abstractions, then the commit and abort rules.

**Motivating example.** A holiday-booking agent reserves a flight, a hotel, and a rental car, then sends a confirmation email (Figure 1). If the flight step fails, the runtime should abort the whole transaction. The four steps have different rollback affordances: the flight hold is refundable, the hotel reservation is compensatable with a cancellation fee, and the confirmation email is irreversible. Reactive compensation cannot gate the email. Under concurrent agents sharing the booking, per-call locks do not span the read-plan-write gap, so stale plans overwrite each other. Under speculative plans, losing branches externalize effects that a snapshot restore cannot roll back. The missing signal is whether the agent work has reached a point where effects can settle safely.

Atomix fills that gap with progress-aware transactions. Inside a transaction, tools run when the agent calls them, but their effects become permanent only when the runtime decides the transaction may settle. Execution, *seal* (freezing the transaction footprint), frontier checking, and commit-or-abort are separate events. Under speculation, each candidate branch follows the same discipline; the controller commits the chosen branch and aborts the others.

The integration layer chooses transaction granularity: the smallest unit of agent intent whose effects must settle together under one invariant. In practice this is an agent action phase, not a single call or an entire long-running workflow.

## 2.1 Core Abstractions

Atomix uses five runtime objects:

- **Epochs** are logical timestamps: totally ordered monotonic scalars in the Naiad sense Murray et al. [2013], assigned by a monotonic allocator. They encode ordering, not identity; a transaction’s identifier (`trace_id`) is orthogonal.
- **Scopes** name resources. A transaction records both read scopes (state the agent observed and depended on) and effect scopes (state the transaction may mutate or release).
- **Effects** describe a tool call’s side effect: its resource scope, an idempotency key (from the tool or adapter), an effect class, and optional release or compensation handlers.
- **Frontiers** are per-resource monotonic cursors on the epoch timeline; the orchestrator advances a resource’s frontier once it certifies that no earlier-epoch work on that resource can still arrive.
- **Transactions** group the read scopes and effects that must settle together. The integration layer declares the granularity. A transaction can span one tool call, one agent read-plan-write phase, one speculative branch, or a short workflow whose effects share one user-facing invariant. Disjunctive “any one of  $K$ ” semantics are not a second transaction shape: each option runs as its own transaction, and the speculation controller commits the chosen branch and aborts the others.

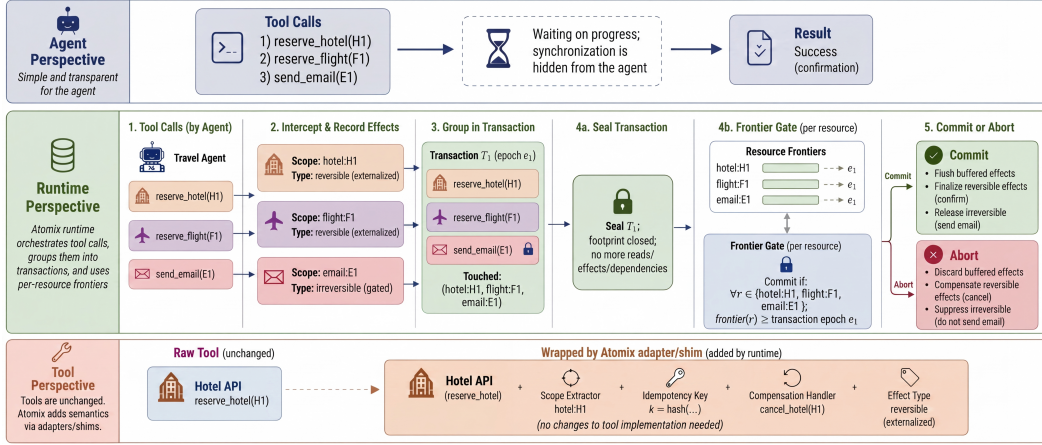


Figure 1: Atomix lifecycle across agent, runtime, and tool layers. The agent receives synchronous tool returns plus one settlement outcome; the runtime records scopes and effects, seals the transaction footprint, gates settlement on per-resource frontiers, and interposes through tool shims.

## 2.2 Lifecycle and Ownership

A transaction moves through four states:

open -> sealed -> waiting.on.progress -> {committed, aborted}

While a transaction is open, adapters may register read scopes and record effects. The orchestrator *seals* the transaction after every in-flight call under that transaction has returned, recorded its effect, or canceled. Seal is a contract: the transaction may not append new read scopes, effect scopes, effects, or dependencies. Atomix enforces the seal by rejecting late additions and aborting the transaction as a protocol violation. A sealed transaction may still wait on frontiers, fail a pre-commit hook, lose branch selection, or abort because a read became stale.

The ownership split is explicit. The agent chooses tool calls; Atomix does not trust it to perform concurrency control. The orchestrator or integration layer chooses transaction granularity, tracks pending calls, seals transactions, selects speculative winners, aborts losers, and advances frontiers. Adapters name scopes, classify effects, derive idempotency keys, and provide release or compensation handlers. The transaction manager enforces the state machine and effect-class settlement. The progress tracker stores frontiers and decides whether a sealed transaction may commit. Semantic validation, when needed, runs in a pre-commit hook.

The progress tracker implements the commit predicate from §2.4 as per-resource frontiers plus waiters keyed by overlapping scopes. A sealed transaction that is not yet admissible remains queued; a frontier advancement wakes affected waiters and re-evaluates the predicate. Frontier advancement is integration-specific: sequential runs advance after each call completes, DAGs after all predecessors on the scope complete, and speculation only after winner selection and loser finalization. Premature advancement is an orchestrator-contract violation; delayed advancement only stalls settlement.

The transaction manager turns this protocol into runtime behavior. `begin` opens a transaction at a fresh epoch; `register_read` records observed scopes; `record_effect` stages the adapter’s effect and updates the transaction’s commit epoch; `seal` freezes the footprint; `request_commit` asks the progress tracker whether settlement is admissible; and `abort` settles without releasing gated effects. If a tool produces a new read or effect after seal, the manager rejects the append and aborts the transaction as a protocol violation. Appendix A gives the full runtime design.

**Worked example.** Figure 1 traces a short booking transaction across the three swim lanes. The reservations (`reserve_hotel/flight/car(d1)`) are reversible and execute eagerly, returning the providers’ real confirmations. Atomix buffers `send_email(d1)` at the adapter gate, gives the agent a synthetic queued-acknowledgment, and fires the call only after per-resource frontiers admit the transaction. **On commit**, Atomix releases the buffered email, finalizes externalized reservations, and the orchestrator returns `committed`. **On abort** (e.g., the flight step fails), compensations cancel the reservations in reverse dependency order and Atomix discards the email. Concurrent rebooking on the same reservation waits for earlier work to finalize. §2.3 details the adapter declarations.

### 2.3 Effect Taxonomy and Tool Adapters

Following classical transaction literature Gray [1978], Atomix classifies effects by *reversibility* (whether compensation can undo the effect) and *externalization timing* (whether the tool commits on call or the runtime can defer it). The runtime maps this classification to three execution paths in Algorithm 1. *Reversible* effects (file edits, DB writes; *reversible-with-cost* variants for refundable purchases) execute eagerly and compensate in reverse dependency order on abort. *Bufferable* effects (local filesystem writes, in-memory state) defer execution, return a staged-acknowledgment, and apply on commit or discard on abort. *Irreversible-gated* effects (emails, transfers, physical actions) hold the call at the adapter, return a queued-acknowledgment, and fire only on commit. An optional fourth class, *idempotent-known-outcome* (e.g., payment-intent APIs with server-side idempotency keys), shares the Reversible path without a custom compensation handler. In the travel example, reservations are Reversible-with-cost and confirmation emails are Irreversible-gated.

Tools integrate through adapters rather than rewrites. To expose a tool to Atomix, the adapter supplies a scope extractor, effect builder, idempotency key, effect class, and optional release or compensation handlers. If the exact resource is unknown before execution, the adapter registers a conservative may-touch scope and refines it after the result identifies concrete resources. Coarse scopes fail closed by reducing concurrency; missing or too-narrow scopes fail open by allowing stale reads or conflicting effects. Appendix Table 4 lists representative adapters.

### 2.4 Commit and Abort Rules

The **commit rule** enforces the progress predicate. For a transaction  $T$ , let  $\mathcal{R}(T)$  be the set of read and effect scopes recorded in its sealed footprint, and let  $e(T)$  be the maximum epoch of those recorded reads and effects. A sealed transaction  $T$  commits only when  $\text{frontier}(r) \geq e(T)$  for every  $r \in \mathcal{R}(T)$ . The orchestrator advances  $\text{frontier}(r)$  to value  $e$  only after every transaction with  $\text{commit\_epoch} < e$  that touches  $r$  has finalized. The strict-less-than makes the predicate non-circular:  $T$  at  $\text{commit\_epoch } e$  commits when  $\text{frontier}(r) \geq e$ , depending only on smaller-epoch peers. In Atomix, commit means final settlement: the runtime releases bufferable and irreversible effects at commit and accepts already-externalized reversible effects as final. Externalized reversible effects may be observed before abort, so Atomix relies on compensation rather than the cursor to undo them.

The **abort rule** replays compensations in reverse dependency order Dunlap et al. [2002]. Abort fires on tool failure, compensation failure, pre-commit veto, losing-branch selection, late post-seal effects, or *stale read* (a waiting transaction whose observed scope an earlier-epoch commit has updated). Atomix tags abort outcomes per effect: *clean* if compensation succeeds; *leaked* if Atomix released any irreversible effect before the abort decision; *unresolved* for compensation failures, residue from reversible-with-cost effects, missing handlers, and *partial-commit* from multiple gated irreversibles where one release succeeded and a later one failed after retry exhaustion (atomic externalization



across heterogeneous endpoints is impossible above the tool layer; §4). Leaked, unresolved, and partial-commit outcomes surface to the operator through the same residue channel.

The commit path also accepts a **pre-commit hook**: a user-supplied predicate over the sealed transaction footprint. Atomix uses it as the attachment point for semantic validation, which the runtime leaves out of scope. Algorithm 2 in Appendix A makes the predicate explicit; shared-resource isolation requires the additional mechanisms in §2.5.

**Visibility model.** Atomix distinguishes *provider visibility* from *Atomix-mediated visibility*. Reversible-eager effects externalize on call, so the provider (and any observer outside Atomix) sees them before commit; abort runs compensation against that already-visible state. Atomix-mediated visibility is what later transactions see through `register_read`: frontiers admit a later  $T'$  only after every smaller-epoch transaction on its scopes has finalized, and abort-on-stale retry rolls  $T'$  back if an observed version is overwritten pre-commit. The composition prevents *Atomix-mediated* dirty reads and stale-write commits but does not undo external mid-transaction visibility of reversible-eager effects. This is the price of supporting unmodified providers behind the shim. Bufferable and irreversible-gated effects have a single visibility event at commit, so the two regimes coincide.

## 2.5 Safety Invariants

**Trusted computing base.** Three assumptions are load-bearing: (i) adapter mediation of every effect-bearing call; (ii) correct adapter metadata (scopes, idempotency keys, compensations, effect classes); (iii) frontier advancement per the strict-less-than contract. Given these, *atomicity by effect class*: bufferable effects are all-or-nothing, reversible effects compensate on abort (residue only on compensation failure), and irreversibles stay gated (§2.3). *Single-process deduplication*: idempotency keys plus durable dedup prevent retry duplicates within one orchestrator process; we do not claim distributed crash-safe exactly-once (Appendix A). *Progress-safe settlement*: frontier ordering composed with scope-on-read and abort-on-stale retry delivers per-resource commit-order settlement; losing-branch bufferable effects leave no persistent state, irreversibles stay gated, and partitions or crashes stall frontiers.

**Implementation.** The Atomix prototype is a  $\sim 2,000$ -line single-process Python runtime; Appendix A gives implementation details and limits.

## 3 Evaluation

We evaluate five questions: **RQ1** fault recovery (§3.2), **RQ2** frontier-gated isolation under contention and speculation (§3.3), **RQ3** irreversible-effect gating (§3.4), **RQ4** differentiation from adjacent systems (§3.5), and **RQ5** runtime cost (Appendix C.5). RQ1–RQ3 mirror the three claims of §1; Appendices B–E give the protocol, supporting tables, boundary controls, and the full adjacent-system matrix.

### 3.1 Methodology

**Workloads, baselines, and statistics.** The primary real-workload fault substrate is  $\tau$ -bench retail (GPT-4.1, `max_steps=30`,  $N=30$  at  $fp \in \{0.10, 0.30\}$ ,  $N=10$  at  $fp=0$ ). WebArena (GPT-5, `reasoning_effort=medium`) and OSWorld (Claude Sonnet 4) are secondary real-infrastructure substrates at  $fp \in \{0.10, 0.30\}$  (Appendix C.1). Multi-agent  $\tau$ -bench uses 2/4/8 agents with GPT-4o-mini for real-LLM validation. Controlled tests isolate individual mechanisms: a combined-stress benchmark ( $N=100$  per cell), a speculation store with  $K \in \{2, 4, 8, 16\}$  branches across four effect classes, contention microbenchmarks, and an irreversible-email task. We focus on benchmarks whose tool calls externalize state rather than reasoning- or capability-centric suites such as SWE-bench Jimenez et al. [2024], GAIA Mialon et al. [2024], or AgentBench Liu et al. [2024]. Our harness implements 9 mechanism-matched baselines (Tx-Full, Tx-NoFrontier+Retry, Saga-Compensation, Checkpoint-Replay, Workflow-Lock, Per-Call-Lock, OCC-Revalidate-and-Retry, TCC-Confirm, Mutex+WAL+Rollback, No-Tx), not full system ports; Mutex+WAL+Rollback tracks SAFEFLOW’s transactional axis without its information-flow control. Faults inject per-call with Bernoulli probability  $fp$  across five classes (F1 pre-execution, F2 post-effect/pre-return, F3 in-compensation, F4 duplicate-delivery, F5 timeout; see Appendix B.2). The injection rate is a stress knob, not a production base rate. Primary metrics are clean task success (RQ1), conflict-serializability violations (RQ2), and classified-before-externalization leak rate (RQ3); we use 95% Clopper-Pearson CIs, report zero-counts as 95% upper bounds, and use Fisher’s exact test for significance. Appendix B gives hardware, substrates, and protocol details.

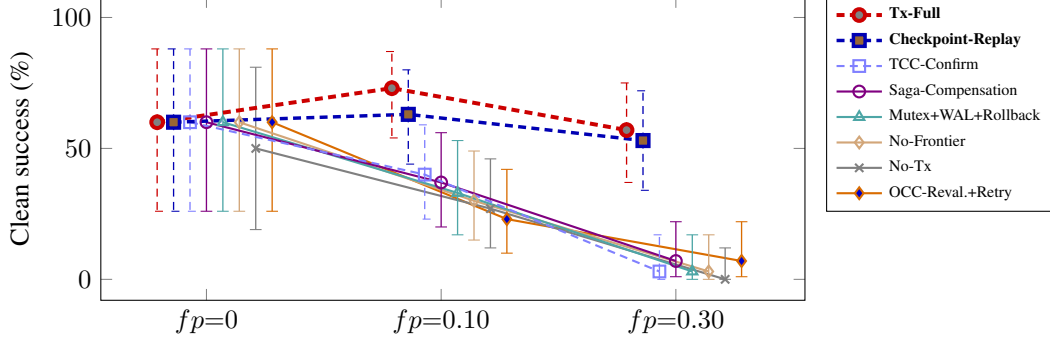


Figure 2: Clean task success on  $\tau$ -bench retail (GPT-4.1, max\_steps=30) across fault rates. Bars show 95% Clopper-Pearson CIs; faulted cells use  $N=30$  and the  $fp=0$  calibration uses  $N=10$ .

### 3.2 RQ1: Fault Recovery on Real LLM-Agent Workloads

**Framing.** RQ1 measures whether a transaction layer that retries faulted calls, compensates externalized reversible effects, and deduplicates ambiguous retries improves end-to-end task success when individual tool calls fail mid-workflow. The clean-success audit checks final state for residue, duplicate effects, and unresolved compensation. The main table compares recovery mechanisms on the same real  $\tau$ -bench sweep. Appendix C.1.1 adds the synthetic cut-point sweep, including Checkpoint-Replay, to localize which fault classes create the separation. RQ1 isolates recovery from isolation: we run mostly sequential workloads and inject faults across the full lifecycle.

**Experimental setup.**  $N=30$  tasks per cell on  $\tau$ -bench retail across  $fp \in \{0, 0.10, 0.30\}$ , comparing Tx-Full against seven baselines (parameters in §3.1). Appendix C.1 adds the WebArena and OSWorld sweeps; Table 8 isolates F2 (post-effect/pre-return) and F4 (duplicate-delivery) faults to localize the mechanism difference.

**Analysis.** Figure 2 gives the real  $\tau$ -bench fault sweep. At  $fp=0.30$ , Tx-Full reaches 57% clean task success, while TCC, Saga, Mutex+WAL+Rollback, No-Frontier, No-Tx, and OCC fall to 0–7%. Checkpoint-Replay is statistically tied on RQ1 (53%, 16/30 vs. Tx-Full’s 17/30, Fisher  $p=1.0$ ; the tie also holds at the full  $N=114$  task pool,  $p \approx 0.50$ , Appendix C.1); RQ3 and combined stress distinguish the two mechanisms. The F2/F4 isolation run in Table 8 shows that the recovery separation is not a duplicate-delivery artifact: idempotent retry handles F4, while F2 post-effect/pre-return failures are where Atomix’s transactional settlement helps most.

### 3.3 RQ2: Frontier-Gated Isolation

**Framing and setup.** Stale-plan races in concurrent agents pass through per-call locks (released between read and write) but force workflow-scoped locks to serialize disjoint work. Atomix’s claim is that frontier ordering plus scope-on-read plus abort-on-stale retry matches workflow-lock safety without that cost. The multi-agent  $\tau$ -bench harness instantiates the booking race on retail orders with two regimes (*forced-overlap*: same order with cancel-vs.-address-modify intents; *disjoint*: different orders), comparing Tx-Full against Workflow-Lock, Per-Call-Lock, OCC, and No-Tx. The synthetic plan-delay variant runs 100 trials/cell at 4 agents forced-overlap and 2/4/8 agents disjoint; the real-LLM variant inserts a GPT-4o-mini call between phases (100 trials at 4 ag, 30 at 8 ag). Appendix C.2 reports speculation isolation on a controlled 4-effect-class benchmark ( $K \in \{2, 4, 8, 16\}$ ).

**Analysis.** Table 1 separates safety, wait, and disjoint-resource throughput. Tx-Full and OCC are the two safety-clean modes with zero wait. Workflow-Lock matches on safety but pays 112 ms wait under contention and 45/105 ms even on disjoint resources. Per-Call-Lock and No-Tx commit 4.9 invariants/run (each one a conflict-cycle witness). OCC sidesteps via rejection at the cost of 3,000 rejected commits per 100 runs at 4 agents. The real-LLM 8-agent block reproduces the qualitative pattern at a higher per-run rate; Tx-Full holds 0 across all configurations. The serializability checker reports no conflict-cycle witnesses for Tx-Full or Workflow-Lock.

**Composition ablation.** An ablation removing each of scope-on-read, abort-on-stale, or per-resource frontiers in turn shows that all three are load-bearing: removing either of the first two collapses to Mutex-Per-Resource (5 invariants/run); removing the third is safety-clean but pays 113 ms wait (Appendix C.2, Table 11).

| Mode                     | Synthetic plan-delay (4 ag, $N=100$ ) |            |           | Real LLM (GPT-4o-mini) |                  | Disjoint wait (ms) |            |
|--------------------------|---------------------------------------|------------|-----------|------------------------|------------------|--------------------|------------|
|                          | Inv./run                              | Wait (ms)  | Blocked   | Inv./run, 4ag          | Inv./run, 8ag    | 4 ag               | 8 ag       |
| <b>Tx-Full</b>           | <b>0.00<sup>†</sup></b>               | <b>0.0</b> | <b>0%</b> | <b>0.0<sup>†</sup></b> | <b>0.0</b>       | <b>0.0</b>         | <b>0.0</b> |
| Workflow-Lock            | 0.00 <sup>†</sup>                     | 112.3      | 75%       | 0.0 <sup>†</sup>       | 0.0              | 44.9               | 104.7      |
| Per-Call-Lock            | 4.91                                  | 0.0        | 0%        | 3.0                    | 7.0              | 0.0                | 0.0        |
| OCC-Revalidate-and-Retry | 0.00 <sup>‡</sup>                     | 0.0        | 0%        | 0.0 <sup>‡</sup>       | 0.0 <sup>‡</sup> | 0.0                | 0.0        |
| No-Tx                    | 4.97                                  | 0.0        | 0%        | 3.0                    | 7.0              | 0.0                | 0.0        |

<sup>†</sup>0 invariant violations and 0 conflict-cycle witnesses across 100 schedule traces (CP-95% upper bound  $\leq 3.7\%$ ). Here, the invariant fires precisely when two committed writes form a conflict cycle on the same scope (cancel commits read-then-writes a state that a concurrent transaction’s read-then-write had already modified), so invariant violations and conflict-cycle witnesses are equal by construction; the upper bound reported here applies to both metrics. <sup>‡</sup>OCC’s zero-invariant rows come from rejecting stale writes rather than committing all work; the per-run rejection rate equals the conflict rate (3.00 OCC rejections/run at 4 agents, 7.00 at 8 agents in the real-LLM substrate, matching the 3.0 and 7.0 invariant violations per run that Per-Call-Lock and No-Tx commit at the same agent counts).

Table 1: Multi-agent  $\tau$ -bench contention results. *Inv./run* is the workload invariant diagnostic; an independent operation log checks conflict-serializability. Wait columns report average blocking time.

| Mode                             | Leaks    | Attempts | Leak rate | 95% CI           |
|----------------------------------|----------|----------|-----------|------------------|
| <b>Tx-Full</b>                   | <b>0</b> | 500      | <b>0%</b> | <b>[0, 0.74]</b> |
| Saga-Compensation                | 400      | 500      | 80%       | [76, 83]         |
| Checkpoint-Replay                | 200      | 500      | 40%       | [36, 44]         |
| TCC-Confirm                      | 0        | 500      | 0%        | [0, 0.74]        |
| Mutex+WAL+Rollback*              | 0        | 500      | 0%        | [0, 0.74]        |
| No-Tx                            | 500      | 500      | 100%      | [99.3, 100]      |
| Atomix-MisclassifiedIrreversible | 300      | 500      | 60%       | [56, 64]         |

Table 2: Irreversible-effect leakage against a real append-only sink. Each row sums 5 abort sources with  $N=100$  trials each, for 500 invalid-send trials per baseline. The final column gives the Clopper-Pearson interval for leak rate.

**Wait scaling.** A global-lock baseline (Tx-GlobalFrontier) pays approximately linear-in- $n$  wait per task at higher agent counts (564 ms at  $n=16$ , 2,369 ms at  $n=64$ ); Tx-Full holds 0 ms wait at every  $n$  via per-resource granularity (Appendix C.2, Table 10).

**Result.** In these trials, the contention checker finds no conflict-cycle witnesses for Tx-Full; per-resource granularity avoids Workflow-Lock’s wait on disjoint resources; and the speculation benchmark reports no persistent losing-branch residue when irreversible effects remain gated.

### 3.4 RQ3: Irreversible-Effect Gating

**Framing.** Effects that cannot be undone once externalized (emails, payments, deletions) fall outside the reach of compensation: a Saga can refund a charge but cannot un-send an email. Gated commit prevents leakage by holding the irreversible effect at the adapter gate until the commit predicate fires. RQ3 asks whether gated commit prevents leakage under enumerated abort sources, and whether effect-class classification costs less developer effort than per-tool TCC handlers or per-resource Mutex+WAL+Rollback gating.

**Experimental setup.** A mixed reversible/irreversible workflow uses a real SMTP/webhook sink, 100 trials per (abort source  $\times$  baseline) cell across 5 abort sources (tool failure, losing speculation, stale read, pre-commit veto, timeout), and 7 baselines. Each row has 500 invalid-send trials and 500 valid-send positive-control trials. The leak metric counts effects classified before externalization that become visible; the harness logs post-externalization residue separately when the provider commits before the adapter records the call (Appendix C.3). Every baseline delivers 500/500 valid sends.

**Analysis.** Table 2 gives invalid-send leakage against the append-only sink. The paired valid-send positive control passes at 500/500 across every baseline, so the gate does not suppress commits that should be released. Three mechanisms hold zero classified-before-externalization leaks: Tx-Full, TCC-Confirm, and Mutex+WAL+Rollback. Saga-Compensation leaks on 4 of 5 abort sources (80%) because compensation cannot un-send; Checkpoint-Replay leaks under tool-failure and timeout (40%) because retried calls re-externalize; the misclassification ablation leaks 60% because the gate never fires. TCC and Mutex+WAL+Rollback match Atomix at zero leaks only because they are fully wired for the email tool here. Under default DB-only wiring they leak the mailbox class on every losing

branch (Appendix C.2). The differentiation is integration cost: each requires per-tool wiring (50 and 30 LOC respectively, scaling linearly with tool count) versus Atomix’s 17 LOC/adaptor regardless of effect class (Appendix C.3, Table 14).

### 3.5 Combined Stress

**Framing.** RQ1–RQ3 each isolate one failure mode. The whole-system claim is the joint case: clean recovery, no irreversible leaks, no stale writes, no losing-branch residue, and no global serialization. Different baselines achieve one axis (TCC and Mutex+WAL+Rollback gate irreversibles; Workflow-Lock and OCC isolate stale plans; Saga and Checkpoint-Replay support recovery), but none covers the union under contention, faults, and irreversible confirmation.

**Setup.** A combined-stress synthetic benchmark exercises all five axes: 4 agents, 2 contended orders (cancel/modify pairs), per-task workflow read  $\rightarrow$  plan-pause  $\rightarrow$  write  $\rightarrow$  irreversible confirmation. Faults inject between write and confirmation with  $f_{\text{class}} \in \{F2, F4, \text{mixed}\}$  at  $fp \in \{0.10, 0.30\}$ . Each run records per-agent clean outcome, invariant violations (cancelled-and-modified on the same order), stale commits, confirmation leaks, duplicates, and wait time. Table 3 defines *run-clean*.

| Mode               | Mixed faults |           | F2-only    | F4-only   | Wait (ms) |
|--------------------|--------------|-----------|------------|-----------|-----------|
|                    | $fp=0.10$    | $fp=0.30$ | $fp=0.10$  | $fp=0.10$ |           |
| <b>Tx-Full</b>     | <b>84</b>    | <b>65</b> | <b>100</b> | 85        | 0.1       |
| OCC                | 82           | 62        | <b>100</b> | 75        | 0.0       |
| Mutex-Workflow     | 81           | 62        | <b>100</b> | 77        | 95.9      |
| TCC-Confirm        | 79           | 63        | <b>100</b> | 81        | 0.1       |
| Mutex+WAL+Rollback | 70           | 24        | 80         | 78        | 0.4       |
| Saga-Compensation  | 68           | 24        | 79         | 87        | 0.1       |
| No-Tx              | 67           | 17        | 86         | 86        | 0.0       |
| Tx-NoAbortOnStale  | 65           | 25        | 83         | 79        | 0.1       |
| Mutex-Per-Resource | 64           | 23        | 85         | 83        | 0.1       |
| Checkpoint-Replay  | 63           | 25        | 81         | 74        | 0.1       |

Table 3: Run-clean rate on combined-stress ( $N=300$  mixed,  $N=100$  F2/F4-only, 4 agents, 2 contended orders). Run-clean requires zero invariant violations, stale commits, confirmation leaks, and duplicates. Wait is from mixed  $fp=0.10$ .

**Analysis.** Tx-Full is Pareto-best on the joint surface. Four mechanisms tie on run-clean rate across both fp regimes (Tx-Full, OCC, Mutex-Workflow, TCC-Confirm; Fisher’s  $p > 0.1$  for every Tx-Full comparison), but each tied baseline pays on a secondary axis: Mutex-Workflow waits 95.9 ms, OCC rejects commits (§3.3), TCC requires per-tool wiring (Appendix C.3); Tx-Full alone reports 0 ms wait, 0 rejections, and a 17-LOC adaptor declaration across effect classes. Tx-NoAbortOnStale at 25–65% is the largest single-primitive ablation drop.

**Real-LLM validation.** Replacing the synthetic plan-pause with a real GPT-4o-mini call between read and write phases reproduces the tier structure (Tx-Full 90%, TCC 93%, Saga 80%, CR 73% run-clean at  $fp=0.10$ ,  $N=30$ ); the wait gap against Mutex-Workflow widens because the global lock blocks every concurrent agent for the full  $\sim 700$  ms LLM call window (Appendix C.2).

**Capability comparison.** RQ1–RQ3 each show one classical mechanism matching Atomix on one safety axis (Workflow-Lock on contention; TCC and Mutex+WAL+Rollback on irreversible-effect leakage). Appendix E (Table 22) gives the union comparison: no compared system covers all seven axes (progress predicates, effect classes, speculation isolation, irreversible gating, stale-plan isolation, shim deployment, single-process deduplication) under shim-only deployment. SAFEFLOW is closest on safety but assumes a co-designed stack with information-flow control; durable-execution engines and watermark systems each cover a subset.

## 4 Discussion and Limitations

**Enforcement boundary and metadata.** Atomix mediates only calls that enter its adaptors; bypass paths lie outside the enforcement boundary. Adapter metadata (scopes, idempotency keys, compensation handlers, effect classes) is safety-critical: too-narrow scopes commit 66/200 invariant violations and a wrong effect class leaks 17/200 irreversibles (Appendix D.5). The released aliasing test suite (Appendix D.4) partially validates this surface; a linter and scope fuzzer remain future work. The orchestrator must also honor the frontier-advancement contract (Appendix D.2).

**Out of scope.** Semantic validation, distributed deployment, and full crash-safe exactly-once are not runtime contributions; the prototype validates single-process deduplication only. Information-flow control and planning-layer validation are orthogonal and compose with Atomix.

**Distributed deployment sketch.** The model appears compatible with distributed implementations: per-resource cursors with monotonic advancement resemble Naiad’s pointstamps Murray et al. [2013] and Flink/Dataflow watermarks Apache Flink [2026], Google Cloud [2026]. A distributed Atomix would require a sharded frontier store with consensus advancement (Calvin-style Thomson et al. [2012]), a replicated effect log for crash-safe exactly-once (Raft, applied to the effect record), and adapter-side idempotency keys preserved across replica failover.

**Atomic externalization across multiple irreversibles.** A transaction with  $\geq 2$  gated irreversibles releases them sequentially; a persistent failure after the first has externalized leaves a partial-commit state. Above the tool layer, the runtime has no way to avoid this: atomic commit across heterogeneous external endpoints requires tool-side TCC or 2PC, which arbitrary REST APIs, SMTP servers, and filesystem syscalls do not provide. Saga, Temporal, SAFEFLOW, and OCC hit the same boundary. Only tool-side commit protocols remove it. Atomix bounds the blast radius via bounded retry, fail-stop on persistent failure, and a partial-commit record naming what externalized and what failed.

**Responsible deployment and liveness.** Atomix reduces accidental leaks but makes adapter mediation and permission boundaries part of the safety case; deployments should combine it with least-privilege tool access, audit logging, and policy checks for irreversible actions. Hot resources serialize by design. Hung agents can stall frontier advancement until the orchestrator times out. Both are deployment concerns rather than runtime bugs.

## 5 Related Work

**Adjacent systems.** Workflow engines and durable-execution platforms (Temporal Temporal Technologies Inc. [2026], AWS Step Functions Amazon Web Services [2026], LangGraph LangChain Inc. [2026], CrewAI CrewAI, Inc. [2026], AutoGen Wu et al. [2024], Inngest Inngest Inc. [2025], Restate Restate Software, Inc. [2026], Cloudflare Workflows Cloudflare, Inc. [2026], DBOS Skadopoulos et al. [2021]) supply durable execution and retries but do not gate external effect settlement on per-resource workflow progress for unmodified tools. The transactional outbox pattern Richardson [2026] is the closest production analog for irreversible-effect gating. Atomix generalizes it to non-database effects (emails, webhooks, physical actions) with adapter-defined keys, plus per-resource progress so multiple agents settle without serializing on a global outbox. SAFEFLOW Li et al. [2025a] is the closest safety comparator but assumes a co-designed stack with information-flow control. AgentGit Li et al. [2025b], GoEX Patil et al. [2024], and Speculative Actions Ye et al. [2026] version state, argue for undo, and predict actions; none supply Atomix’s cross-call progress signal. SagaLLM Chang and Geng [2025], ALAS Geng and Chang [2025], MAS-FIRE Jia et al. [2026], PEARL Wang et al. [2026], Sherlock Ro et al. [2025], and AsyncLM Gim et al. [2024] address planning-layer validation, async calls, and agent-induced fault characterization; they compose with Atomix through the pre-commit hook.

**Novelty.** Atomix composes Sagas Garcia-Molina and Salem [1987], Try-Confirm-Cancel Pardon and Pautasso [2014], write-ahead logging Mohan et al. [1992], escrow locks O’Neil [1986], deterministic scheduling Thomson et al. [2012], and streaming watermarks Murray et al. [2013], Akidau et al. [2021], Apache Flink [2026], Google Cloud [2026], Chandy and Lamport [1985] into a four-event split (execute, seal, frontier-check, settle) at the tool boundary. Each event depends on a different ingredient: adapters classify effects, the transaction manager seals footprints, the progress tracker advances per-resource frontiers, and effect-class settlement chooses between flush, gate-fire, and accept-as-final. In our comparison, each classical primitive omits or conflates at least one ingredient at the tool boundary (Appendix E.1, Table 23; full matrix in Table 22).

## 6 Conclusion

LLM agents mutate external state, but current orchestrators cannot transactionally settle tool effects under faults, speculation, contention, and irreversibility. Atomix adds sealed footprints, per-resource progress, scope-on-read with abort-on-stale retry, and effect-class-aware settlement behind shim adapters; the composition closes a gap each prior mechanism covers only partially. Empirically, Atomix is Pareto-best on the joint surface: top run-clean tier at zero wait, zero rejections, and 17 LOC per adapter. Distributed deployment remains future work.

## References

- Tyler Akidau, Edmon Begoli, Slava Chernyak, Fabian Hueske, Kathryn Knight, Kenneth Knowles, Daniel Mills, and Dan Sotolongo. Watermarks in stream processing systems: Semantics and comparative analysis of Apache Flink and Google Cloud Dataflow. *Proceedings of the VLDB Endowment*, 14(12):3135–3147, 2021. doi: 10.14778/3476311.3476389.
- Amazon Web Services. What is Step Functions? <https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>, 2026. Accessed 2026-05-07.
- Anthropic. Claude Code overview. <https://code.claude.com/docs/en/overview>, 2026. Accessed 2026-05-07.
- Apache Flink. Stateful stream processing. <https://nightlies.apache.org/flink/flink-docs-release-2.2/docs/concepts/stateful-stream-processing/>, 2026. Accessed 2026-05-07.
- K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985. doi: 10.1145/214451.214456.
- Edward Y. Chang and Longling Geng. SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning. *Proceedings of the VLDB Endowment*, 18(12):4874–4886, 2025. doi: 10.14778/3750601.3750611.
- Cloudflare, Inc. Workflows, 2026. URL <https://developers.cloudflare.com/agents/concepts/workflows/>. Accessed 2026-05-07.
- CrewAI, Inc. CrewAI documentation. <https://docs.crewai.com/>, 2026. Accessed 2026-05-07.
- George W. Dunlap, Samuel T. King, Sukru Cinar, Murtaza A. Basrai, and Peter M. Chen. ReVirt: Enabling intrusion analysis through virtual-machine logging and replay. In *Proceedings of the 5th Symposium on Operating Systems Design and Implementation (OSDI '02)*, pages 211–224, Boston, MA, 2002. USENIX Association. doi: 10.1145/1060289.1060309.
- Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data (SIGMOD '87)*, pages 249–259. ACM, 1987. doi: 10.1145/38713.38742.
- Longling Geng and Edward Y. Chang. ALAS: Transactional and dynamic multi-agent LLM planning. *arXiv preprint arXiv:2511.03094*, 2025. doi: 10.48550/arXiv.2511.03094. URL <https://arxiv.org/abs/2511.03094>.
- In Gim, Seung-seob Lee, and Lin Zhong. Asynchronous LLM function calling. *arXiv preprint arXiv:2412.07017*, 2024. doi: 10.48550/arXiv.2412.07017. URL <https://arxiv.org/abs/2412.07017>.
- Google Cloud. Exactly-once in Dataflow. <https://cloud.google.com/dataflow/docs/concepts/exactly-once>, 2026. Accessed 2026-05-07.
- J. N. Gray. Notes on data base operating systems. In R. Bayer, R. M. Graham, and G. Seegmüller, editors, *Operating Systems: An Advanced Course*, volume 60 of *Lecture Notes in Computer Science*, pages 393–481. Springer, Berlin, Heidelberg, 1978. ISBN 978-3-540-08755-7. doi: 10.1007/3-540-08755-9\_9.
- Inngest Inc. How Inngest functions are executed: Durable execution, 2025. URL <https://www.inngest.com/docs/learn/how-functions-are-executed>. Accessed 2026-05-07.
- Jin Jia, Zhiling Deng, Zhuangbin Chen, Yingqi Wang, and Zibin Zheng. MAS-FIRE: Fault injection and reliability evaluation for LLM-based multi-agent systems. *arXiv preprint arXiv:2602.19843*, 2026. doi: 10.48550/arXiv.2602.19843. URL <https://arxiv.org/abs/2602.19843>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.

- LangChain Inc. LangGraph overview. <https://docs.langchain.com/oss/python/langgraph/overview>, 2026. Accessed 2026-05-07.
- Peiran Li, Xinkai Zou, Zhuohang Wu, Ruifeng Li, Shuo Xing, Hanwen Zheng, Zhikai Hu, Yuping Wang, Haoxi Li, Qin Yuan, Yingmo Zhang, and Zhengzhong Tu. SAFEFLOW: A principled protocol for trustworthy and transactional autonomous agent systems. *arXiv preprint arXiv:2506.07564*, 2025a. doi: 10.48550/arXiv.2506.07564. URL <https://arxiv.org/abs/2506.07564>.
- Yang Li, Siqi Ping, Xiyu Chen, Xiaojian Qi, Zigan Wang, Ye Luo, and Xiaowei Zhang. AgentGit: A version control framework for reliable and scalable LLM-powered multi-agent systems. *arXiv preprint arXiv:2511.00628*, 2025b. doi: 10.48550/arXiv.2511.00628. URL <https://arxiv.org/abs/2511.00628>.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=zAdUBOaCTQ>.
- Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for General AI assistants. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=fibxvahvs3>.
- Model Context Protocol Project. Model context protocol specification, version 2025-11-25. <https://modelcontextprotocol.io/specification/2025-11-25>, 2025. Accessed 2026-05-07.
- C. Mohan, Don Haderle, Bruce G. Lindsay, Hamid Pirahesh, and Peter M. Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992. doi: 10.1145/128765.128770.
- Derek Gordon Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martín Abadi. Naiad: A timely dataflow system. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP '13)*, pages 439–455. ACM, 2013. doi: 10.1145/2517349.2522738.
- Patrick E. O’Neil. The escrow transactional method. *ACM Transactions on Database Systems*, 11(4): 405–430, 1986. doi: 10.1145/7239.7265.
- OpenAI. Function calling — OpenAI API. <https://developers.openai.com/api/docs/guides/function-calling>, 2026. Accessed 2026-05-07.
- Guy Pardon and Cesare Pautasso. Atomic distributed transactions: A RESTful design. In *Proceedings of the 23rd International Conference on World Wide Web (WWW '14 Companion), 5th International Workshop on Web APIs and RESTful Design (WS-REST 2014)*, pages 943–948. ACM, 2014. doi: 10.1145/2567948.2579221.
- Shishir G. Patil, Tianjun Zhang, Vivian Fang, Noppapon C., Roy Huang, Aaron Hao, Martin Casado, Joseph E. Gonzalez, Raluca Ada Popa, and Ion Stoica. GoEX: Perspectives and designs towards a runtime for autonomous LLM applications. *arXiv preprint arXiv:2404.06921*, 2024. doi: 10.48550/arXiv.2404.06921. URL <https://arxiv.org/abs/2404.06921>.
- Restate Software, Inc. Welcome to Restate!, 2026. URL <https://docs.restate.dev/>. Accessed 2026-05-07.
- Chris Richardson. Pattern: Transactional outbox, 2026. URL <https://microservices.io/patterns/data/transactional-outbox.html>. Accessed 2026-05-07.
- Yeonju Ro, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, Aditya Akella, Zhangyang Wang, Mattan Erez, and Esha Choukse. Sherlock: Reliable and efficient agentic workflow execution. *arXiv preprint arXiv:2511.00330*, 2025. doi: 10.48550/arXiv.2511.00330. URL <https://arxiv.org/abs/2511.00330>.

- Athinagoras Skiadopoulos, Qian Li, Peter Kraft, Kostis Kaffes, Daniel Hong, Shana Mathew, David Bestor, Michael Cafarella, Vijay Gadepally, Goetz Graefe, Jeremy Kepner, Christos Kozyrakis, Tim Kraska, Michael Stonebraker, Lalith Suresh, and Matei Zaharia. DBOS: A DBMS-oriented operating system. *Proceedings of the VLDB Endowment*, 15(1):21–30, 2021. doi: 10.14778/3485450.3485454.
- Temporal Technologies Inc. Temporal Docs — Temporal Platform Documentation. <https://docs.temporal.io/>, 2026. Accessed 2026-05-07.
- Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J. Abadi. Calvin: Fast distributed transactions for partitioned database systems. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data (SIGMOD '12)*, pages 1–12. ACM, 2012. doi: 10.1145/2213836.2213838.
- Qihao Wang, Mingzhe Lu, Jiayue Wu, Yue Hu, and Yanbing Liu. PEARL: Plan exploration and adaptive reinforcement learning for multihop tool use. In Yi Mei, Chao Qian, Quan Bai, Bing Xue, and Sankalp Khanna, editors, *PRICAI 2025: Trends in Artificial Intelligence – 22nd Pacific Rim International Conference on Artificial Intelligence, Wellington, New Zealand, November 17–21, 2025, Proceedings, Part IV*, volume 16454 of *Lecture Notes in Computer Science*, pages 639–651, Wellington, New Zealand, 2026. Springer, Singapore. doi: 10.1007/978-981-95-7081-2\_44.
- Gerhard Weikum. Principles and realization strategies of multilevel transaction management. *ACM Transactions on Database Systems*, 16(1):132–180, 1991. doi: 10.1145/103140.103145.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling (COLM 2024)*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>.
- Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. Speculative actions: A lossless framework for faster AI agents. In *The Fourteenth International Conference on Learning Representations (ICLR 2026)*. OpenReview.net, 2026. URL <https://openreview.net/forum?id=P0G0k5ws1g>. Oral presentation.



## Appendix

### A Runtime Model and Implementation

This appendix gives the model and implementation details behind §2. Atomix sits on the tool-call path between the orchestrator and external resources. The runtime requires orchestrator hooks for logical-time allocation, transaction sealing, branch selection, and progress advancement, but tools need no modification when all calls pass through adapters.

#### A.1 Lifecycle and Commit Predicate

Algorithm 1 gives the runtime lifecycle for one tool invocation. Tool execution records a footprint, SEAL freezes that footprint, and REQUESTCOMMIT waits for the progress predicate before releasing gated effects.

**Algorithm 1.** *Runtime lifecycle for a tool invocation. The classification step routes reversible effects through eager execution, bufferable effects through deferred execution, and irreversible effects through a held adapter gate.*

```
1:  $adapter \leftarrow \text{LOOKUPADAPTER}(tool)$ 
2:  $scopes \leftarrow adapter.SCOPEs(args)$ 
3:  $epoch \leftarrow \text{NEXT EPOCH}(trace\_id)$ 
4:  $tx \leftarrow \text{BEGIN OR ATTACH}(scopes, epoch)$ 
5:  $\text{REGISTER READ}(tx, scopes)$  // records observed version per scope (Alg. 3)
6:  $cls \leftarrow adapter.EFFECT CLASS(args)$ 
7: if  $cls \in \{\text{REVERSIBLE}, \text{REVERSIBLE WITH COST}\}$  then
8:    $result \leftarrow \text{EXECUTE}(tool, args)$  // eager: tool runs now
9:    $effect \leftarrow adapter.TOEFFECT(args, result, epoch, cls)$ 
10: else if  $cls = \text{BUFFERABLE}$  then
11:    $effect \leftarrow adapter.TODEFERRED(args, epoch, cls)$ 
12:    $result \leftarrow \text{STAGED ACK}(effect)$  // tool not yet executed; flushed on commit
13: else
14:    $effect \leftarrow adapter.TOGATED(args, epoch, cls)$ 
15:    $result \leftarrow \text{QUEUED ACK}(effect)$ 
16: end if
17:  $\text{RECORD EFFECT}(tx, effect)$  // updates  $tx.commit\_epoch$ 
18: if  $\text{READY TO SEAL}(tx)$  then
19:    $\text{SEAL}(tx)$ 
20:    $\text{REQUEST COMMIT}(tx)$ 
21: end if
22: return  $result$ 
23:
24: procedure  $\text{REQUEST COMMIT}(tx)$ 
25: if  $tx$  is not sealed then
26:    $\text{ABORT}(tx, \text{protocol\_violation})$ 
27: else if not  $\text{CAN COMMIT}(tx)$  then
28:    $\text{QUEUE UNTIL READY}(tx)$ 
29: else if  $\text{READ IS STALE}(tx)$  then
30:    $\text{ABORT}(tx, \text{stale\_read}); \text{SCHEDULE RETRY}(tx)$  // Alg. 3
31: else if not  $\text{PRE COMMIT OK}(tx)$  then
32:    $\text{ABORT}(tx, \text{pre\_commit\_veto})$ 
33: else
34:    $\text{APPLY EFFECTS ONCE}(tx.effects)$ 
35:    $\text{MARK COMMITTED}(tx)$ 
36: end if
37:
38: procedure  $\text{APPLY EFFECTS ONCE}(effects)$ 
39: for all  $e \in effects$  ordered by recorded epoch do
40:   if  $e.cls = \text{BUFFERABLE}$  then
41:      $\text{FLUSH BUFFERED}(e)$  // execute the deferred tool call
42:   else if  $e.cls = \text{IRREVERSIBLE}$  then
```

```

43:   FIREGATED( $e$ )    // release the adapter gate; tool runs now
44:   else
45:     ACCEPTASFINAL( $e$ )    // already externalized at execution
46:   end if
47: end for
48:
49: procedure ABORT( $tx, reason$ )
50: for all  $e \in \text{REVERSE}(tx.effects)$  do
51:   if  $e.cls = \text{BUFFERABLE}$  then
52:     DISCARDBUFFERED( $e$ )    // tool never ran; nothing to undo
53:   else if  $e.cls = \text{IRREVERSIBLE}$  then
54:     CLEARGATE( $e$ )    // tool never ran; nothing to undo
55:   else if  $e$  has compensation and was applied then
56:     COMPENSATE( $e$ )
57:   end if
58: end for
59: MARKABORTED( $tx, reason$ )

```

Algorithm 2 makes the progress predicate explicit. The predicate is meaningful only after seal: before then, the read and effect footprint can still grow. Frontier values are non-decreasing. The orchestrator advances  $\text{frontier}(r)$  to value  $e$  only after every transaction with  $\text{commit\_epoch} < e$  on  $r$  has finalized. The strict-less-than makes the predicate non-circular: a transaction at  $\text{commit\_epoch } e$  commits when  $\text{frontier}(r) \geq e$ , which depends only on smaller-epoch peers, not on itself. Premature advancement is a safety violation; delayed advancement stalls settlement.

**Algorithm 2.** *Progress-safe commit predicate. The ADVANCEFRONTIER precondition is the orchestrator's contract; the runtime fails closed if the contract is violated.*

```

1: State:  $\text{frontier}[r] \leftarrow 0$  for each resource  $r$ 
2:
3: function CANCOMMIT( $tx$ ):
4:    $e \leftarrow tx.commit\_epoch$     // max over recorded read and effect epochs
5:   for all  $r \in tx.read\_scopes \cup tx.effect\_scopes$  do
6:     if  $\text{frontier}[r] < e$  then
7:       return false
8:     end if
9:   end for
10:  return true
11:
12: procedure ADVANCEFRONTIER( $r, e$ ):
13: precondition: every transaction with  $\text{commit\_epoch} < e$  that touches  $r$  has finalized
14:   $\text{frontier}[r] \leftarrow \max(\text{frontier}[r], e)$ 

```

Algorithm 3 gives the stale-read detection invoked from REQUESTCOMMIT. Each resource carries a monotonic version counter incremented on every committed write. REGISTERREAD stores the version at read time, and READISSTALE fails the predicate if any observed version has advanced before the transaction reaches commit. Stale-read aborts trigger a retry on the new state. Without this primitive, frontier ordering alone permits a sealed transaction to commit after a smaller-epoch peer has overwritten its snapshot.

**Algorithm 3.** *Stale-read detection and retry. Combined with frontier ordering and scope registration, this delivers per-resource commit-order settlement under contention.*

```

1: State:  $\text{version}[r] \leftarrow 0$  for each resource  $r$ 
2:
3: procedure REGISTERREAD( $tx, scopes$ ):
4:   for all  $r \in scopes$  do
5:      $tx.read\_versions[r] \leftarrow \text{version}[r]$     // observed version at first read
6:   end for
7:
8: function READISSTALE( $tx$ ):
9:   for all  $(r, v) \in tx.read\_versions$  do

```

```

10:  if version[r]  $\neq$  v then
11:    return true
12:  end if
13: end for
14: return false
15:
16: procedure ONCOMMIT(tx): // called by APPLYEFFECTSONCE
17: for all r  $\in$  tx.effect_scopes do
18:   version[r]  $\leftarrow$  version[r] + 1
19: end for

```

## A.2 Runtime Components

The runtime has three components. *Tool adapters* translate heterogeneous tool I/O into scoped reads and typed effects. The *progress tracker* stores per-scope frontiers and waiters, then decides when sealed transactions may settle. The *transaction manager* enforces the lifecycle and routes effects by class.

Atomix canonicalizes scopes before frontier checks. Two scopes overlap iff they share a type prefix and one canonical path is a segment-level prefix of the other, or matches through a declared wildcard. If the exact resource is unknown before execution, the adapter registers a conservative may-touch scope and refines it after the result identifies concrete resources. Coarse scopes fail closed by reducing concurrency; missing or too-narrow scopes fail open by allowing stale reads or conflicting effects.

Frontier advancement is integration-specific. Sequential runs advance after each call completes; DAGs advance when all predecessors on the scope complete; speculation advances only after the controller chooses the winning branch and every losing branch has finalized. Because the signal is resource-local, a stalled scope does not delay unrelated work.

The manager routes effects by class because no single recovery action fits all tools. Bufferable effects are held in a per-transaction staging area invisible to other transactions; reversible externalized effects execute immediately but register compensation; irreversible effects remain pending until commit. If a tool produces a new read or effect after seal, the manager rejects the append and aborts the transaction as a protocol violation.

## A.3 Prototype and Adapter Surface

The Atomix prototype is a  $\sim$ 2,000-line single-process Python library. Transactions, scopes, effects, frontiers, and seal state are dataclasses; the transaction manager and progress tracker run as coroutines on the orchestrator’s event loop. The durable path records effect-log entries and idempotency keys, using JSON Lines for trace analysis and SQLite-backed deduplication for committed effects. Unit tests validate crash-safe deduplication on this single-process path; distributed deployment requires a shared frontier store and replicated effect log.

Tools integrate through adapters rather than rewrites. Each adapter supplies a scope extractor, effect builder, idempotency key, and optional release or compensation handlers. Table 4 lists representative adapters used by the experiments.

| Workload / adapter       | Representative scope(s)                                        | Idempotency key pattern                                          | Compensation / gating                                                                     |
|--------------------------|----------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Filesystem writes        | absolute path,<br>/repo/src/app.py                             | path:trace:<br>epoch:branch                                      | restore previous contents;<br>delete files created by aborted<br>writes                   |
| WebArena actions         | page scope,<br>webarena:site:page<br>optional element suffix   | webarena:action:<br>trace:epoch:branch                           | no generic per-action compen-<br>sation; settlement controlled<br>by transaction scopes   |
| OSWorld GUI ac-<br>tions | UI app scope,<br>osworld:ui:app:<br>text_input or mouse:region | action:args:<br>trace:epoch:branch                               | action-specific compensation<br>when available; read-only ac-<br>tions carry no scope     |
| $\tau$ -bench tools      | real runner:<br>tau2:tool<br>harness:<br>retail:order:id       | real runner:<br>trace:epoch:tool<br>harness:<br>order-scoped key | structured tool calls are re-<br>playable; multi-agent harness<br>checks order invariants |

Table 4: Adapter specification examples. Scope granularity and compensation are adapter-defined.

## B Evaluation Protocol and Limitations

This appendix gives the common protocol behind §3. Results in Appendix C use this protocol unless a subsection states a narrower controlled substrate.

### B.1 Workloads, Baselines, and Metrics

Main-text workloads use  $\tau$ -bench retail (GPT-4.1, `max_steps=30`,  $N=30$  for faulted cells and  $N=10$  for the no-fault calibration), multi-agent  $\tau$ -bench at 2/4/8 agents, the combined-stress benchmark, and controlled speculation, contention, and irreversible-effect benchmarks. WebArena (GPT-5 with `reasoning_effort=medium`, `max_steps=30`) and OSWorld (Claude Sonnet 4) provide secondary real-infrastructure sweeps at  $fp \in \{0.10, 0.30\}$  (Appendix C.1, Table 5).

Modes are **Tx-Full**, **Tx-NoFrontier+Retry**, **Saga-Compensation**, **Checkpoint-Replay**, **Workflow-Lock**, **Per-Call-Lock**, **OCC-Revalidate-and-Retry**, **TCC-Confirm**, **Mutex+WAL+Rollback**, and **No-Tx**. Each RQ uses only the baselines that directly test its mechanism claim.

Metrics follow the main text: clean task success, standard task success, leaked or unresolved residue, duplicate effects, conflict-serializability violations, committed writes, wait, blocked-rate, throughput, and storage overhead. Confidence intervals are 95% exact intervals for proportions and bootstrap intervals for continuous quantities when reported.

### B.2 Fault Injection

The harness injects faults per tool call using a Bernoulli profile with probability  $fp$ . An injected fault raises an exception at the tool interface. Tx-Full aborts the affected transaction and retries with a fresh epoch when retry is enabled. Checkpoint-Replay restores workflow state and replays, which can re-fire externalized calls. No-Tx models a lost response: the tool may have already applied the action, but the caller sees an error. This captures timeouts and dropped acknowledgements where the caller cannot distinguish failure from partial externalization.

### B.3 Evaluation Limitations

These limits scope what the empirical results support, beyond the discussion in §4.

- **Distributed deployment.** Experiments assume the orchestrator advances frontiers correctly within one process; a distributed deployment needs a shared frontier store and replicated effect log (§4). Unit tests validate crash-safe deduplication only on the single-process SQLite path.
- **Speculation coverage.** Speculation-contamination results include bufferable, filesystem, database-row, and mailbox sinks. Tools without compensation handlers can still leave unresolved residue.
- **Real-workload contention.** The real  $\tau$ -bench cells are largely sequential; we exercise multi-agent contention on the synthetic plan-delay harness with a real-LLM validation block. Broader real-world multi-agent and speculative benchmarks remain future work.
- **Baseline fidelity.** Baselines are mechanism-matched implementations inside a common harness, not full ports of any production system; the fidelity disclosure in §3.1 explains the trade-off.

### B.4 Reproducibility Manifest

**Models and snapshots.** GPT-4.1 (gpt-4.1-2025-04-14) for  $\tau$ -bench retail; GPT-5 (gpt-5-2025-08-07) with `reasoning_effort=medium` for WebArena; Claude Sonnet 4 (claude-sonnet-4-20250514) for OSWorld; GPT-4o-mini (gpt-4o-mini-2024-07-18) for the multi-agent  $\tau$ -bench real-LLM substrate. All accessed via the providers’ standard chat-completions APIs.

**Workload versions and task lists.**  $\tau$ -bench retail commit tau-bench@main (vendored at submission time); WebArena commit webarena@1.0; OSWorld commit osworld@v1.1. The artifact’s tasks/ directory lists the exact  $\tau$ -bench retail task IDs used per cell, the WebArena task pool (10 tasks), and the OSWorld task subset (7 tasks). Cells use the same task IDs across baselines so success rates compare like-for-like.

**Hyperparameters.** Agent decoding is deterministic where possible: temperature 0 for  $\tau$ -bench (GPT-4.1), default for GPT-5 (which sets its own under `reasoning_effort=medium`), temperature 0 for OSWorld (Claude Sonnet 4) and the multi-agent substrate (GPT-4o-mini). `max_steps` is benchmark-specific (30 for  $\tau$ -bench retail and WebArena, 50 and 100 for OSWorld). Retry budget per faulted call is 3 with exponential backoff (50 ms initial,  $1.5\times$  multiplier, capped at 1 s).

**Seeds and stochasticity.** The fault injector and the synthetic plan-delay harness use a deterministic per-cell seed derived from (benchmark, mode, fp, run\_index). The harness logs replayable schedule-trace ordering across agents; the conflict-serializability checker reads from this independent log.

**Data schema.** Each run emits `events.jsonl` (every adapter call with epoch, scope, effect class, idempotency key, return), `commits.jsonl` (commit/abort outcomes with reason and residue tags), and `summary.csv` (one row per run with the metrics named in the tables). The conflict-serializability checker reads `events.jsonl` and emits `cycles.jsonl` listing any conflict-cycle witnesses.

**Reproduction commands.** The artifact ships a Makefile with one target per main-text cell (make `rq1-multirate`, make `rq2-multiagent`, make `rq3-irrev`, make `joint-surface`) plus appendix targets, a `check-cycles` target that runs the serializability checker over all collected logs, and a `regenerate-tables.py` script that consumes `summary.csv` and re-emits the LaTeX tables in the paper.

**Hardware.** Single Linux workstation, Intel Xeon (8 cores), 32 GiB RAM, 1 TiB NVMe. WebArena runs on a self-hosted Docker swarm of 6 replicas (Reddit, Gitlab, Magento Admin, OneStopShop, Map, Shopping); OSWorld runs in KVM Ubuntu-22.04 VMs (4 vCPU, 8 GiB RAM, 24 GiB qcow2 image);  $\tau$ -bench runs in-process; the multi-agent SQLite store is on the local NVMe.

## C Additional Results by Research Question

This appendix groups supporting results in the same order as the evaluation questions in §3.

### C.1 RQ1 Fault Recovery Details

Tables 5 and 6 give the real-benchmark clean-success details behind RQ1. The  $\tau$ -bench sweep is the primary RQ1 signal; Table 5 gives the corresponding WebArena and OSWorld sweeps at  $fp=0.10$  and  $fp=0.30$ . The qualitative pattern matches  $\tau$ -bench: Tx-Full and Checkpoint-Replay have the highest rates, Mutex+WAL+Rollback is the nearest other baseline, and the remaining recovery-only and isolation-only baselines drop sharply.

| Mode                     | WebArena                         |                                  | OSWorld                          |                                  |
|--------------------------|----------------------------------|----------------------------------|----------------------------------|----------------------------------|
|                          | $fp=0.10$                        | $fp=0.30$                        | $fp=0.10$                        | $fp=0.30$                        |
| <b>Tx-Full</b>           | <b>73.0 <math>\pm</math> 5.5</b> | <b>57.2 <math>\pm</math> 6.7</b> | <b>63.0 <math>\pm</math> 7.5</b> | <b>37.0 <math>\pm</math> 8.8</b> |
| Checkpoint-Replay        | 68.0 $\pm$ 4.8                   | 53.2 $\pm$ 4.3                   | 62.0 $\pm$ 6.2                   | 37.1 $\pm$ 5.1                   |
| TCC-Confirm              | 25.0 $\pm$ 4.2                   | 18.0 $\pm$ 3.5                   | 10.0 $\pm$ 3.0                   | 6.0 $\pm$ 2.8                    |
| Saga-Compensation        | 18.0 $\pm$ 3.2                   | 10.0 $\pm$ 2.8                   | 7.0 $\pm$ 2.8                    | 4.0 $\pm$ 2.2                    |
| Mutex+WAL+Rollback       | 65.0 $\pm$ 5.1                   | 48.0 $\pm$ 5.5                   | 55.0 $\pm$ 6.8                   | 32.0 $\pm$ 6.5                   |
| No-Frontier              | 17.0 $\pm$ 3.8                   | 7.8 $\pm$ 2.0                    | 7.0 $\pm$ 2.5                    | 0.5 $\pm$ 0.5                    |
| No-Tx                    | 13.0 $\pm$ 3.5                   | 3.3 $\pm$ 2.4                    | 3.0 $\pm$ 1.8                    | 0.0 $\pm$ 0.0                    |
| OCC-Revalidate-and-Retry | 30.0 $\pm$ 4.5                   | 22.0 $\pm$ 4.0                   | 12.0 $\pm$ 3.2                   | 9.0 $\pm$ 3.2                    |

Table 5: Clean task success on WebArena and OSWorld at  $fp \in \{0.10, 0.30\}$ . Values are percentages with 95% Clopper-Pearson CI half-widths. WebArena uses GPT-5 (reasoning\_effort=medium, max\_steps=30); OSWorld uses Claude Sonnet 4.

| Mode                     | $\tau$ -bench ( $fp=0$ , $N=10$ ) | $\tau$ -bench ( $fp=0.10$ , $N=30$ ) | $\tau$ -bench ( $fp=0.30$ , $N=30$ ) |
|--------------------------|-----------------------------------|--------------------------------------|--------------------------------------|
| <b>Tx-Full</b>           | 60 [26, 88]                       | <b>73 [54, 87]</b>                   | <b>57 [37, 75]</b>                   |
| Checkpoint-Replay        | 60 [26, 88]                       | 63 [44, 80]                          | 53 [34, 72]                          |
| TCC-Confirm              | 60 [26, 88]                       | 40 [23, 59]                          | 3 [0, 17]                            |
| Saga-Compensation        | 60 [26, 88]                       | 37 [20, 56]                          | 7 [1, 22]                            |
| Mutex+WAL+Rollback       | 60 [26, 88]                       | 33 [17, 53]                          | 3 [0, 17]                            |
| No-Frontier              | 60 [26, 88]                       | 30 [15, 49]                          | 3 [0, 17]                            |
| No-Tx                    | 50 [19, 81]                       | 27 [12, 46]                          | 0 [0, 12]                            |
| OCC-Revalidate-and-Retry | 60 [26, 88]                       | 23 [10, 42]                          | 7 [1, 22]                            |

Table 6: Clean task success on  $\tau$ -bench retail (GPT-4.1, max\_steps=30). Values are percentages with 95% Clopper-Pearson intervals.

**Tx-Full vs. Checkpoint-Replay at the full task pool ( $N=114$ ).** The  $N=30$  multirate run above uses the first 30 of the 38 retail tasks. Extending to the full task pool with 3 trials per task ( $N=114$ ) gives Tx-Full  $67/114 = 58.8\%$  (CP-95% [49.4, 67.7]%) and Checkpoint-Replay  $61/114 = 53.5\%$  (CP-95% [43.9, 63.0]%). The two are statistically tied at this rate (Fisher’s exact two-sided  $p \approx 0.50$ ). The 8 additional tasks, plus the additional trials of the first 30, are harder than the first-30 cohort, dropping both rates by  $\sim 10$ – $15$ pp relative to the  $fp=0.10$  first-30 cells. Tx-Full and Checkpoint-Replay are tied on RQ1 task-success alone at  $fp=0.10$  across the full task pool. Their differentiation appears in RQ3 (irreversible leak rate: 0/500 vs. 200/500, Table 2) and in the joint-surface combined-stress benchmark (Table 3), where Checkpoint-Replay’s retry path re-externalizes the irreversible confirmation and falls to 25% run-clean at  $fp=0.30$  vs Tx-Full’s 65%.

**Bursty-fault sensitivity.** To test whether the qualitative pattern survives correlated faults, we re-ran the  $\tau$ -bench retail sweep with a burst-mode injector at marginal  $fp \approx 0.12$  (fault-entry probability 0.04, burst length 3,  $N=30$ ). Tx-Full leads at  $20/30 = 67\%$ , with Checkpoint-Replay at 57% and TCC-Confirm/No-Tx tied at 60% (Table 7). The qualitative tier structure carries over from the Bernoulli case, but absolute differences narrow because correlated triples mean the workload sees fewer distinct fault episodes per task ( $\sim 1.2$  bursts/task vs.  $\sim 3$  independent Bernoulli faults at the same marginal rate), giving recovery-only mechanisms fewer chances to fail.

| Mode                     | Bursty (entry=0.04, len=3) | Bernoulli ( $fp=0.10$ ) |
|--------------------------|----------------------------|-------------------------|
| <b>Tx-Full</b>           | <b>20/30 = 67%</b>         | <b>22/30 = 73%</b>      |
| Checkpoint-Replay        | 17/30 = 57%                | 19/30 = 63%             |
| TCC-Confirm              | 18/30 = 60%                | 12/30 = 40%             |
| No-Tx                    | 18/30 = 60%                | 8/30 = 27%              |
| Mutex+WAL+Rollback       | 15/30 = 50%                | 10/30 = 33%             |
| No-Frontier              | 14/30 = 47%                | 9/30 = 30%              |
| Saga-Compensation        | 13/30 = 43%                | 11/30 = 37%             |
| OCC-Revalidate-and-Retry | 12/30 = 40%                | 7/30 = 23%              |

Table 7: Bursty-fault sensitivity on  $\tau$ -bench retail at marginal  $fp \approx 0.12$ ,  $N=30$ . The Bernoulli column repeats  $fp=0.10$  from Table 6 for direct comparison.

### C.1.1 Fault-Class Cut-Point Sweep

**F2 vs. F4 isolation.** Tx-Full’s lead at  $fp=0.10$  could come from F4 duplicate delivery, where idempotency keys alone suffice, rather than the F2 post-effect/pre-return cell that the model section identifies as load-bearing. Table 8 separates the two cases. We force-fired only F2 (`f2_share_of_exception=1.0`, no duplicates) and only F4 (`duplicate_probability=0.10`, no exceptions) in separate  $\tau$ -bench runs at  $fp=0.10$ ,  $N=30$ , across all 7 baselines. In the F4-only column, baselines cluster at 53–73%; in the F2-only column, Tx-Full is 6–43pp higher than every non-Atomix baseline.

| Mode                     | F2-only ( $fp=0.10$ ) | F4-only ( $fp=0.10$ ) | $\Delta$ (F2–F4) |
|--------------------------|-----------------------|-----------------------|------------------|
| <b>Tx-Full</b>           | <b>63 [44, 80]</b>    | 67 [47, 83]           | –3pp             |
| Checkpoint-Replay        | 57 [37, 75]           | 57 [37, 75]           | 0pp              |
| Saga-Compensation        | 43 [25, 63]           | 70 [51, 85]           | –27pp            |
| OCC-Revalidate-and-Retry | 40 [23, 59]           | 53 [34, 72]           | –13pp            |
| Mutex+WAL+Rollback       | 33 [17, 53]           | 57 [37, 75]           | –23pp            |
| No-Frontier              | 37 [20, 56]           | 67 [47, 83]           | –30pp            |
| TCC-Confirm              | 20 [8, 39]            | 63 [44, 80]           | –43pp            |
| No-Tx                    | 30 [15, 49]           | 73 [54, 87]           | –43pp            |

Table 8: Fault-class isolation on  $\tau$ -bench retail at  $fp=0.10$ ,  $N=30$  per cell, with 95% Clopper-Pearson CIs. F2-only forces post-effect/pre-return faults; F4-only injects silent duplicate delivery with no exceptions.

Table 9 expands the three-tier synthetic-substrate sweep by fault class. The synthetic harness uses  $fp \in \{0.02, 0.10, 0.30\}$  rather than the body’s  $fp \in \{0, 0.10, 0.30\}$  because  $fp=0.02$  models a low-fault tier where most cells saturate at 100% and the F4 column differentiates the mechanisms.

| Mode                     | $fp=0.02$  |            |            |            |            | $fp=0.10$  |            |            |            |            | $fp=0.30$ |           |           |            |           |
|--------------------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|------------|-----------|-----------|-----------|------------|-----------|
|                          | F1         | F2         | F3         | F4         | F5         | F1         | F2         | F3         | F4         | F5         | F1        | F2        | F3        | F4         | F5        |
| <b>Tx-Full</b>           | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> | <b>100</b> | <b>96</b> | <b>98</b> | <b>96</b> | <b>100</b> | <b>96</b> |
| Tx-NoFrontier+Retry      | 100        | 100        | 100        | 88         | 100        | 100        | 100        | 100        | 64         | 100        | 94        | 98        | 96        | 16         | 96        |
| Saga-Compensation        | 100        | 100        | 100        | 92         | 100        | 100        | 100        | 100        | 60         | 100        | 96        | 98        | 94        | 17         | 96        |
| Checkpoint-Replay        | 100        | 100        | 100        | 88         | 100        | 100        | 100        | 100        | 56         | 100        | 98        | 97        | 98        | 16         | 96        |
| OCC-Revalidate-and-Retry | 100        | 100        | 100        | 90         | 100        | 100        | 100        | 100        | 66         | 100        | 96        | 97        | 98        | 18         | 94        |
| No-Tx                    | 88         | 89         | 94         | 94         | 90         | 60         | 56         | 58         | 59         | 54         | 20        | 14        | 17        | 12         | 16        |

Table 9: Clean-success sweep by fault class on the synthetic substrate. Cells use  $fp \in \{0.02, 0.10, 0.30\}$ ; F3 is compensation failure and F4 is duplicate delivery.

## C.2 RQ2 Isolation and Speculation Details

Table 11 isolates parts of the composition needed for stale-plan isolation. Table 12 checks losing-branch residue across effect classes. Table 13 sweeps resource-granularity throughput across overlap fractions  $o$ ; at full overlap ( $o=1$ ), Tx-Full averages 0.60 wait barriers per transaction (p95 = 1.00) versus 1.00 for Tx-GlobalFrontier and Workflow-Lock and 0.00 for the unsafe No-Tx baseline. No-Tx’s throughput decreases with  $o$  because the workload itself serializes writes on shared hot keys, not because of coordination wait.

**Wait scaling at higher agent counts.** The 4- and 8-agent disjoint cells in Table 1 understate the cost of Workflow-Lock-style global-frontier coordination. Re-running the same disjoint workload at  $n \in \{16, 32, 64\}$  agents on the synthetic substrate ( $N=30$  runs each, plan-pause 50 ms) gives Tx-GlobalFrontier (the per-resource-granularity ablation, equivalent to a single global lock for the entire workflow): 564 ms / 1,163 ms / 2,369 ms wait per task at  $n=16/32/64$ , while Tx-Full holds 0 ms wait across every  $n$  (Table 10). Wait grows roughly linearly with agent count past  $n=8$ ; at  $n=64$ , a single global lock spends 2.4 seconds blocked per tool call. For workflows with  $\sim 10$  tool calls per task, this compounds to  $\sim 24$  seconds of lock wait per workflow at  $n=64$  vs. 0 ms with per-resource granularity.

| Mode                                   | $n=2$      | $n=4$      | $n=8$      | $n=16$     | $n=32$     | $n=64$     |
|----------------------------------------|------------|------------|------------|------------|------------|------------|
| <b>Tx-Full</b>                         | <b>0.0</b> | <b>0.0</b> | <b>0.0</b> | <b>0.0</b> | <b>0.0</b> | <b>0.0</b> |
| Tx-GlobalFrontier (single global lock) | 14.8       | 44.9       | 104.7      | 563.9      | 1,163.2    | 2,368.6    |

Table 10: Average wait (ms) per task on the disjoint multi-agent benchmark. Cells use  $N=100$  for  $n \in \{2, 4, 8\}$  and  $N=30$  for  $n \in \{16, 32, 64\}$ . Tx-GlobalFrontier replaces per-resource coordination with one global lock.

**Ablation interpretation.** Each row removes exactly one composition primitive from Tx-Full and reruns the same workload. The clean ablation requires a per-phase locking pattern (lock the resource for the read, release during plan-pause, re-acquire for the write); abort-on-stale fires on the second-phase re-acquire if the OCC version counter has advanced. Tx-NoScopeOnRead skips the read-phase lock; Tx-NoAbortOnStale takes the read-phase lock but skips the version check at write; Tx-GlobalFrontier replaces the per-resource lock with a single global one. The first two converge to Mutex-Per-Resource behavior (5/run violations from stale-overwrite); the third is safety-clean but throughput-bound. No single primitive suffices: Tx-NoScopeOnRead races on the read, Tx-NoAbortOnStale commits stale writes, and Tx-GlobalFrontier blocks every disjoint pair. Tx-Full’s 0 violations and 0 ms wait require all three.

| Composition ablation     | Inv./run    | Wait (ms)  | Extra writes (per run) | What it removes                               |
|--------------------------|-------------|------------|------------------------|-----------------------------------------------|
| <b>Tx-Full (control)</b> | <b>0.00</b> | <b>0.0</b> | <b>0.0</b>             | none                                          |
| Tx-NoScopeOnRead         | 4.83        | 0.0        | 6.5                    | scope registration on first read              |
| Tx-NoAbortOnStale        | 4.87        | 0.0        | 6.5                    | abort-on-stale retry                          |
| Tx-GlobalFrontier        | 0.00        | 113.0      | 0.0                    | per-resource granularity (single global lock) |

Table 11: Composition ablations on multi-agent  $\tau$ -bench, 4 agents forced-overlap,  $N=100$ . *Inv./run*: invariant violations; *Wait*: average lock wait; *Extra writes*: writes beyond 10 per run.

| Mode                             | Buffer   | Filesystem | DB rows  | Mailbox     |
|----------------------------------|----------|------------|----------|-------------|
| <b>Tx-Full</b>                   | <b>0</b> | <b>0</b>   | <b>0</b> | <b>0</b>    |
| Tx-GlobalFrontier                | 0        | 0          | 0        | 0           |
| Mutex+WAL+Rollback               | 0        | 0          | 0        | 0           |
| TCC-Confirm                      | 0        | 0          | 0        | <b>1000</b> |
| Saga-Compensation                | 0        | 0          | 0        | <b>1000</b> |
| OCC-Revalidate-and-Retry         | 0        | 0          | 0        | <b>1000</b> |
| Atomix-MisclassifiedIrreversible | 0        | 0          | 0        | <b>1000</b> |

Table 12: Speculation contamination across four effect classes, averaged over  $K \in \{2, 4, 8, 16\}$  ( $N=200$  trials per  $K$ ). Cells report persistent loser-branch effects per 1,000 attempted transactions; the mailbox column is the irreversible class.

TCC-Confirm, OCC-Revalidate-and-Retry, and Saga-Compensation use default DB-only wiring in this controlled benchmark: try/cancel and rejection apply only to the DB-row class, leaving the mailbox class ungaurded. The RQ3 benchmark in Table 2 uses a fully wired TCC-Confirm adapter for the email tool, so TCC reaches 0/500 leaks there.

**Per- $K$  speculation scaling.** Across  $K \in \{2, 4, 8, 16\}$  ( $N=200$  trials per cell), losing-branch residue on the irreversible mailbox class falls into two patterns: Tx-Full, Tx-GlobalFrontier, and Mutex+WAL+Rollback hold 0 leaks per 1,000 attempts at every  $K$ ; TCC-Confirm, OCC-Revalidate-and-Retry, Saga-Compensation, and Atomix-MisclassifiedIrreversible scale linearly at 200/600/1400/3000 (one losing branch per  $K-1$  losers).

| Mode              | $o=0$ disjoint | $o=0.25$ | $o=0.5$ | $o=0.75$ | $o=1$ overlap |
|-------------------|----------------|----------|---------|----------|---------------|
| <b>Tx-Full</b>    | <b>5.09</b>    | 4.43     | 3.22    | 2.28     | 1.66          |
| Tx-GlobalFrontier | 1.00           | 1.00     | 1.00    | 1.00     | 1.00          |
| Workflow-Lock     | 1.00           | 1.00     | 1.00    | 1.00     | 1.00          |
| No-Tx             | 1000           | 873      | 751     | 627      | 503           |

Table 13: Resource-granularity throughput sweep, in transactions per step.  $o$  is the fraction of overlapping scopes; No-Tx is an unsafe upper bound, not a correctness-preserving baseline.

### C.3 RQ3 Irreversible-Effect and Residue Details

**Integration cost vs. leak prevention.** TCC-Confirm and Mutex+WAL+Rollback tie Atomix at 0/500 leaks (Table 2); integration cost separates them. Table 14 compares the per-baseline mechanism size and per-tool wiring each requires.

| Mechanism                | tx_manager LOC<br>(one-time) | Per-tool wiring<br>(per integrated tool) | Leaks/500 | Composes with<br>R/I/S         |
|--------------------------|------------------------------|------------------------------------------|-----------|--------------------------------|
| <b>Tx-Full</b>           | <b>500</b> (runtime)         | <b>17</b> (scope+effect builder)         | <b>0</b>  | <b>R/I/S</b>                   |
| TCC-Confirm              | 115                          | ~50 (try/confirm/cancel triple)          | 0         | I only                         |
| Mutex+WAL+Rollback       | 168                          | ~30 (explicit gating predicate)          | 0         | I + S                          |
| OCC-Revalidate-and-Retry | 135                          | ~15 (read-version site)                  | 500*      | S only                         |
| Saga-Compensation        | 92                           | ~30 (compensation handler)               | 400       | R only (irreversibles leak)    |
| Checkpoint-Replay        | N/A <sup>†</sup>             | N/A <sup>†</sup>                         | 200       | R only (retry re-externalizes) |

\*OCC has no irreversible-gating component; the retry path re-externalizes on every abort source, so the expected leak rate matches No-Tx ( $500/500 = 100\%$ ) by construction. <sup>†</sup>Checkpoint-Replay does not decompose into a transaction manager plus per-tool adapter; it snapshots and replays whole-process state, so the LOC columns do not apply.

Table 14: RQ3 integration cost versus leak prevention. *tx\_manager LOC* is one-time harness integration; *Per-tool wiring* is per integrated tool. *Composes with* marks recovery (R), irreversible gating (I), and stale-plan isolation (S).

**Prototype-scale cost.** The per-tool gap compounds across an actual deployment surface. The Atomix prototype ships 20 adapters at ~17 LOC of declarations per adapter (Appendix D.5), totalling ~340 LOC. A TCC-Confirm-style port of the same surface would require ~50 LOC per tool of bespoke try/confirm/cancel handlers, or ~1,000 LOC over 20 tools. Mutex+WAL+Rollback’s explicit per-effect gating wiring would add ~30 LOC per tool, or ~600 LOC for the same surface. Atomix’s per-tool cost is the same scope+key+class declaration regardless of effect class; TCC’s and Mutex+WAL+Rollback’s are class-specific and grow with the share of irreversible tools.



**Per-abort-source breakdown.** Table 15 breaks down irreversible leaks by abort source. The claim is conditional on the irreversible effect being classified before externalization. Provider-side commits that happen before the adapter records the effect are outside the preventive guarantee; the runtime surfaces them as unresolved residue when it detects them.

| Mode                             | Tool fail | Losing spec. | Stale read | Pre-commit veto | Timeout  |
|----------------------------------|-----------|--------------|------------|-----------------|----------|
| <b>Tx-Full</b>                   | <b>0</b>  | <b>0</b>     | <b>0</b>   | <b>0</b>        | <b>0</b> |
| Saga-Compensation                | 1000      | 1000         | 1000       | 0               | 1000     |
| Checkpoint-Replay                | 1000      | 0            | 0          | 0               | 1000     |
| TCC-Confirm                      | 0         | 0            | 0          | 0               | 0        |
| Mutex+WAL+Rollback*              | 0         | 0            | 0          | 0               | 0        |
| No-Tx                            | 1000      | 1000         | 1000       | 1000            | 1000     |
| Atomix-MisclassifiedIrreversible | 0         | 1000         | 1000       | 0               | 1000     |

Table 15: Irreversible-effect leaks by abort source. Cells report classified-before-externalization leaks per 1,000 attempts.

**Compensation-failure classification.** §2.4 states that abort outcomes are tagged per effect (*clean*, *leaked*, *unresolved*, *partial-commit*). Operators rely on these tags to triage residue. Table 16 verifies the tagging itself: under injected compensation failures at rate  $cf_p$ , does the runtime’s clean/unresolved classification match a synthetic ground-truth oracle? With residue classification enabled (Tx-Full rows), 0/600 mismatches at every  $cf_p$ . With it disabled (Tx-Full-NoResidueClassification at  $cf_p=0.3$ ), 36/600 transactions reported clean when ground truth said unresolved. The runtime exposes that silent failure mode by default. The result does not claim that failed compensation leaves no residue; it claims that the residue-tagging mechanism accurately reports which transactions did and did not.

| Mode                            | $cf_p$ | Trials | GT clean | Classified clean | Mismatch  |
|---------------------------------|--------|--------|----------|------------------|-----------|
| <b>Tx-Full</b>                  | 0.0    | 600    | 600      | 600              | <b>0</b>  |
| <b>Tx-Full</b>                  | 0.1    | 600    | 589      | 589              | <b>0</b>  |
| <b>Tx-Full</b>                  | 0.3    | 600    | 557      | 557              | <b>0</b>  |
| Saga-Compensation               | 0.3    | 600    | 559      | 559              | 0         |
| Retry-only                      | 0.3    | 600    | 534      | 534              | 0         |
| Tx-Full-NoResidueClassification | 0.3    | 600    | 564      | 600              | <b>36</b> |

Table 16: Compensation-failure classification. *GT clean* is oracle residue status; *Mismatch* counts classification errors.

#### C.4 Joint Correctness Surface Details

**Real-LLM combined-stress.** The synthetic combined-stress tier structure (§3.5, Table 3) reproduces under a real GPT-4o-mini call inserted between the read and write phases. At  $fp=0.10$  mixed,  $N=30$  on the same 4-agent / 2-order workload: Tx-Full 27/30 = 90% run-clean (635 ms wait per run), TCC-Confirm 28/30 = 93% (statistically tied with Tx-Full at this  $N$ ), Saga-Compensation 24/30 = 80%, Checkpoint-Replay 22/30 = 73%, Mutex-Workflow 22/30 = 73% but at 1,651 ms wait per run. The wait gap against Mutex-Workflow widens under real LLM thinking time because the global lock blocks every concurrent agent for the full  $\sim 700$  ms GPT-4o-mini call window. The tier structure and the cost differentiation carry over from the synthetic harness.

#### C.5 RQ5 Overhead and Storage

RQ5 bounds the cost of mediating every tool call. Tx-Full adds  $7.7 \mu\text{s}$  per step against  $0.8 \mu\text{s}$  for No-Tx, below 0.01% of typical tool latency (50 ms–10 s). Real  $\tau$ -bench at  $fp=0$  runs in 163.9 s for Tx-Full and 171.2 s for No-Tx, so the wrapper does not increase end-to-end time in that cell. Coordination cost appears only on Workflow-Lock: it pays  $\sim 112$  ms wait per task at 4 agents forced-overlap and  $\sim 45$  ms even on disjoint resources, while Tx-Full pays 0 ms in both regimes (Table 1). Uncoordinated modes pay zero wait but produce the invariant violations in the same table. CPython garbage-collection pauses cause the p99 spike at  $K=16$  in Table 17.

Table 17 reports speculation commit latency. Effect-log storage from replaying real-run traces totals 1.25 MB across 15 JSONL files ( $\sim 75$  bytes/entry).

| Branches ( $K$ ) | Setup ( $\mu s$ ) | Commit ( $\mu s$ ) | Total / p99 ( $\mu s$ ) |
|------------------|-------------------|--------------------|-------------------------|
| 2                | 13                | 85                 | 98 / 295                |
| 4                | 23                | 142                | 165 / 273               |
| 8                | 44                | 321                | 365 / 1339              |
| 16               | 86                | 939                | 1025 / 23198            |

Table 17: Speculation commit latency ( $n = 200$  runs per  $K$ ). At  $K = 16$ , total frontier-gating overhead is about 1 ms.

## D Boundary Conditions and Metadata Sensitivity

These limits govern Atomix’s claims. They correspond to the discussion of semantic correctness, frontier contracts, crash scope, scope metadata, and adapter annotations in §4.

### D.1 Semantic-Validation Boundary

Atomix does not reject semantically invalid commits on its own. Across 22 invalid attempts, Tx-Full alone commits 22/22; with the pre-commit hook installed, Tx-Full + SemanticHook commits 0/22. Semantic validation belongs in the pre-commit hook.

### D.2 Frontier-Contract Boundary

Table 18 verifies that safety depends on correct frontier advancement. Stalled advancement blocks progress; premature advancement admits out-of-order commits.

| Frontier-advancement mode                          | Out-of-order commits | Invariant violations | Tx-Full waits               |
|----------------------------------------------------|----------------------|----------------------|-----------------------------|
| Correct (advances after each commit finalizes)     | <b>0/200</b>         | <b>0/200</b>         | 0                           |
| Stalled (no advancement)                           | 0/200                | 0/200                | <b>200/200</b> (100% block) |
| Premature (advance before earlier-epoch finalizes) | <b>200/200</b>       | 200/200              | 0                           |

Table 18: Frontier-contract negative control on one resource. Premature advancement causes out-of-order commits; stalled advancement blocks progress.

### D.3 Crash-Window Boundary

Table 19 enumerates the prototype’s single-process crash windows for the SQLite-backed adapter. Duplicate effects remain zero in this unit-test scope; we do not claim distributed crash safety.

| Kill point                                | Trials | Duplicate effects | Lost effects |
|-------------------------------------------|--------|-------------------|--------------|
| (i) before pending-log write              | 50     | <b>0</b>          | 50           |
| (ii) after pending-log, before effect     | 50     | <b>0</b>          | 50           |
| (iii) after effect, before committed-mark | 50     | <b>0</b>          | 0            |
| (iv) after committed-mark, before retry   | 50     | <b>0</b>          | 0            |

Table 19: Crash-window enumeration for the SQLite-backed adapter. Duplicate effects remain zero; lost effects occur only before the effect runs.

### D.4 Scope Canonicalization and Aliasing

The serializability checker reads scope IDs from an independent log path and cross-checks them against the alias suite summarized in Table 20. The suite covers filesystem paths,  $\tau$ -bench entities, browser DOM selectors, and read-write dependencies.

| Scope strategy           | Should-conflict cases<br>(across all substrates) | Missed conflicts<br>(false neg.) | False conflicts<br>(false pos.) |
|--------------------------|--------------------------------------------------|----------------------------------|---------------------------------|
| <b>Tx-Full canonical</b> | 28                                               | <b>0</b>                         | <b>0</b>                        |
| Tx-NaiveStringScopes     | 28                                               | <b>12</b>                        | 0                               |
| Coarse-Global            | 28                                               | 0                                | <b>28</b>                       |
| Fine-NoOverlapAware      | 28                                               | <b>12</b>                        | 0                               |

Table 20: Aliasing and canonicalization stress across four substrates. Missed conflicts are false negatives; false conflicts are false positives.

## D.5 Annotation-Error Sensitivity

Atomix’s correctness depends on adapter-supplied metadata: scope extractors, idempotency-key derivers, compensation handlers, and effect classifications. Table 21 characterizes which annotation errors fail closed and which fail open. Across the 20 prototype adapters, the annotation surface totals 134 LOC of scope extractors and 197 LOC of effect builders, or 331 LOC overall (16.6 LOC/adapter mean).

| Configuration               | Inv./200  | Leaked irrev./200 | Failure mode            |
|-----------------------------|-----------|-------------------|-------------------------|
| Tx-Full (control)           | 0         | 0                 | none                    |
| E10-OB Over-broad scope     | <b>0</b>  | <b>0</b>          | failure-closed          |
| E10-TN Too-narrow scope     | <b>66</b> | 0                 | failure-open            |
| E10-WC Wrong effect class   | 0         | <b>17</b>         | failure-open            |
| E10-NC Missing compensation | 0         | <b>35</b>         | failure-open with alert |

Table 21: Annotation-error sensitivity. Cells report invariant violations and leaked irreversibles per 200 trials.

Atomix is failure-closed only for over-broad annotations. Too-narrow scopes, wrong effect classes, and missing compensation are operator responsibilities. The runtime surfaces unresolved residue when it can, but it enforces the metadata it is given.

## E Adjacent Systems and Related Work Details

This appendix expands the RQ4 comparison from §3.5. Table 22 compares systems on the axes that load-bear in RQ1–RQ3. ✓ denotes primary support in the cited publication or documentation; footnotes mark partial support.

| System                                                                          | ATOMIX'S AXES       |                     |                        |                      |                        |                     |                            | ORTHOGONAL |                    |
|---------------------------------------------------------------------------------|---------------------|---------------------|------------------------|----------------------|------------------------|---------------------|----------------------------|------------|--------------------|
|                                                                                 | <i>Per-resource</i> | <i>Effect-class</i> | <i>Spec. isolation</i> | <i>Irrev. gating</i> | <i>Stale-plan iso.</i> | <i>Shim deploy.</i> | <i>Single-proc. dedup.</i> | <i>IPC</i> | <i>Sem. valid.</i> |
| <b>Atomix (this paper)</b>                                                      | ✓                   | ✓                   | ✓                      | ✓                    | ✓                      | ✓                   | ✓                          | ✗          | ✗                  |
| SAFEFLOW Li et al. [2025a]                                                      | ✗                   | ✗                   | ✗                      | ✓ <sup>a</sup>       | ✓                      | ✗                   | ✓                          | ✓          | ✗                  |
| SagaLLM Chang and Geng [2025]                                                   | ✗                   | ✗                   | ✗                      | ✗                    | ✓                      | ✗                   | ✗                          | ✗          | ✓                  |
| ALAS Geng and Chang [2025]                                                      | ✗                   | ✗                   | ✗                      | ✗                    | ✓                      | ✗                   | ✗                          | ✗          | ✓                  |
| Temporal Saga Temporal Technologies Inc. [2026], Garcia-Molina and Salem [1987] | ✗                   | ✗                   | ✗                      | ✗ <sup>b</sup>       | ✗                      | ✓                   | ✓                          | ✗          | ✗                  |
| LangGraph checkpointer LangChain Inc. [2026]                                    | ✗                   | ✗                   | ✗                      | ✗                    | ✗                      | ✓                   | ✓                          | ✗          | ✗                  |
| AgentGit Li et al. [2025b]                                                      | ✗                   | ✗                   | ✗                      | ✗                    | ✗                      | ✓                   | ✗                          | ✗          | ✗                  |
| GoEX Patil et al. [2024]                                                        | ✗                   | ✓ <sup>c</sup>      | ✗                      | ✓ <sup>c</sup>       | ✗                      | ✓                   | ✗                          | ✗          | ✗                  |
| Speculative Actions Ye et al. [2026]                                            | ✗                   | ✗                   | ✗                      | ✗                    | ✗                      | ✓                   | ✗                          | ✗          | ✗                  |
| Classical Saga Garcia-Molina and Salem [1987]                                   | ✗                   | ✗                   | ✗                      | ✗                    | ✗                      | ✓                   | ✗                          | ✗          | ✗                  |
| Classical OCC Weikum [1991]                                                     | ✗                   | ✗                   | ✗                      | ✗                    | ✓ <sup>d</sup>         | ✓                   | ✗                          | ✗          | ✗                  |
| Streaming watermarks Murray et al. [2013], Akidau et al. [2021]                 | ✓                   | ✗                   | ✗                      | ✗                    | ✗                      | ✗                   | ✓                          | ✗          | ✗                  |

<sup>a</sup> Irreversible handling depends on SAFEFLOW's co-designed execution protocol, not shim-only deployment. <sup>b</sup> Temporal can implement application-specific gates in workflow code, but does not provide a generic settlement gate for unmodified external tools. <sup>c</sup> GoEX discusses undo and damage confinement where tool support exists; it does not provide Atomix's cross-call progress predicate. <sup>d</sup> OCC detects stale writes on versioned resources but rejects or retries work and does not handle irreversible-effect gating.

Table 22: Capability comparison across load-bearing axes. Superscripts mark partial support or the limitation discussed in text.

The comparison separates Atomix's execution-layer scope from orthogonal information-flow and semantic-validation systems. SAFEFLOW is the strongest transactional comparator but assumes a co-designed stack with information-flow control. SagaLLM and ALAS add semantic validation. Temporal and LangGraph provide durable orchestration but no per-resource settlement gate for unmodified tool effects. Streaming watermarks provide progress signals for records, not for branch-selected tool effects. Atomix's single-process deduplication entry is limited to the prototype path described in Appendix A; it is not a distributed exactly-once claim.

## E.1 Four-Ingredient Composition Matrix

Table 23 summarizes which classical mechanisms supply each of Atomix's four ingredients. The matrix focuses on the four-event split rather than the broader axes of Table 22: sealed transaction footprints, per-resource progress predicates, scope-on-read with abort-on-stale retry, and effect-class-aware settlement.

| Mechanism                                                          | Sealed footprint     | Per-resource progress | Scope-on-read + abort-on-stale | Effect-class settlement |
|--------------------------------------------------------------------|----------------------|-----------------------|--------------------------------|-------------------------|
| Sagas Garcia-Molina and Salem [1987]                               | $\times$             | $\times$              | $\times$                       | $\times$                |
| TCC Pardon and Pautasso [2014]                                     | partial <sup>a</sup> | $\times$              | $\times$                       | partial <sup>b</sup>    |
| OCC Weikum [1991]                                                  | $\times$             | $\times$              | $\checkmark$                   | $\times$                |
| Two-phase locking Mohan et al. [1992]                              | $\times$             | $\times$              | $\times$                       | $\times$                |
| Watermarks (Naiad/Flink) Murray et al. [2013], Apache Flink [2026] | $\times$             | $\checkmark^c$        | $\times$                       | $\times$                |
| Workflow-Lock                                                      | $\times$             | $\times$              | $\times$                       | $\times$                |
| Durable execution (Temporal/Inngest/Restate/DBOS)                  | partial <sup>d</sup> | $\times$              | $\times$                       | $\times$                |
| Transactional outbox Richardson [2026]                             | $\times$             | $\times$              | $\times$                       | partial <sup>e</sup>    |
| SAFEFLOW Li et al. [2025a]                                         | $\checkmark$         | partial <sup>f</sup>  | partial <sup>f</sup>           | partial <sup>f</sup>    |
| <b>Atomix</b>                                                      | $\checkmark$         | $\checkmark$          | $\checkmark$                   | $\checkmark$            |

<sup>a</sup>TCC’s try-phase reservations freeze a partial footprint but do not name read scopes or seal cross-call dependencies.

<sup>b</sup>TCC distinguishes try, confirm, and cancel handlers per tool but does not classify effects across a runtime taxonomy.

<sup>c</sup>Watermarks track per-key progress for data records, not for branch-selected tool effects. <sup>d</sup>Durable-execution engines persist workflow state but do not freeze a footprint of read scopes and effect scopes for cross-call settlement. <sup>e</sup>Outbox tables gate one externalization channel (the message broker) idempotently but do not handle reversible effects, speculation residue, or cross-call progress. <sup>f</sup>SAFEFLOW assumes a co-designed stack with information-flow control; its progress, scope-on-read, and effect-class equivalents are tied to that stack rather than provided to unmodified tool adapters.

Table 23: Four-ingredient composition matrix. Atomix exposes all four ingredients to unmodified tool adapters as a single runtime contract.